*** Files (aka PGDATA)**
65.1 Database File layout: https://www.postgresql.org/docs/current/storage-file-layout.html

PGDATA aka base data directory
Directory containing files for a single **database cluster**.
Defined by **PGDATA variable** or **-D option** of server

e.g. find
check process

```
$ echo $PGDATA

$ ps ax | grep postgres

24383   ??  S      0:00.06 /opt/homebrew/opt/
postgresql@17/bin/postgres -D /opt/homebrew/var/
postgresql@17
```

check **data_directory** config

```
> SHOW data_directory
         data_directory
_____________________________________
 /opt/homebrew/var/postgresql@17
```

files
<PGDATA>/postgresql.conf config
<PGDATA>/postgresql.auto.conf
<PGDATA>/pg_ident.conf
<PGDATA>/pg_hba.conf

<PGDATA>/PG_VERSION
<PGDATA>/base/ per-database data
...

*** Tablespace**
https://www.postgresql.org/docs/current/manage-ag-tablespaces.html

Tablespace
A custom **physical storage** i.e. way to store database objects other than the default **PGDATA**.
Main uses
1) **disk partition**: allow database objects to be managed on separate partitions
2) **performance**: e.g. store high usage tables on fast storage, and low usage tables on cheaper storage.

*** Logging**
Logging: https://www.postgresql.org/docs/current/runtime-config-logging.html

Logging
logging_collector
By default, logs are printed only to **stderr** of the server process.
i.e. NOT written to file

2 ways to capture logs
1) **syslog**

2) **logging_collector**: writes log to a file
Disabled by default.
MUST be

usage: check if file logging is enabled

1) Check settings
Check **logging_collector** is enabled

```
> show logging_collector

 logging_collector                              -- on if
enabled
____________________
 on

-- Check PGDATA
> show data_directory;                          -- PGDATA
                                                -- Hence
logs are in
         data_directory                         --   /
opt/homebrew/var/postgresql@17/log/
postgresql-2025-01-01-000000.log
_____________________________________
 /opt/homebrew/var/postgresql@17
```

2) Get actual log path

Check **<PGDATA>/current_logfiles**

```
# e.g. if PGDATA=/opt/homebrew/var/postgresql@17
$ cd /opt/homebrew/var/postgresql@17

# Get log logfile path
$ cat current_logfiles
stderr /opt/homebrew/var/postgresql@17/log/
postgresql-2025-04-29_184524.log

# See log
$ less /opt/homebrew/var/postgresql@17/log/
postgresql-2025-04-29_184524.log
```

usage: enable file logging with **logging_collector**

Step 1: enable **logging_collector** config

in <PGDATA>/postgresql.conf

```
log_destination = 'stderr'        # Valid values are
combinations of
                      # stderr, csvlog, jsonlog, syslog,
and
                      # eventlog, depending on platform.
                      # csvlog and jsonlog require
                      # logging_collector to be on.

# This is used when logging to stderr:
logging_collector = on        # Enable capturing of
stderr, jsonlog,
                      # and csvlog into log files.
Required
                      # to be on for csvlogs and
jsonlogs.
                      # (change requires restart)

# These are only used if logging_collector is on:
log_directory = 'log'         # directory where log
files are written,
                      # can be absolute or relative to
PGDATA
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'# log
file name pattern,
```

Step 2: restart PostgreSQL
e.g. on macos

```
$ brew service stop postgresql@17
$ brew service start postgresql@17
```

Step 3: check log

```
$ cd <PGDATA>/log

$ less +F postgresql-2025-04-14_112233.log
```

*** usage: find where database cluster files are located**
2 ways to define the base directory:
1) **PGDATA**=<dir> environment variable

2) **postgres -D <dir>** option of the server CLI

e.g.

```
# case 1: if defined by PGDATA
$ echo $PGDATA
/opt/homebrew/var/postgresql@17
```

```
# case 2: if defined by -D flag
$ ps aux | grep postgres

yuinishizawa      1744   0.0  0.0 411223040   2800   ??
Ss    5:41pm   0:00.02 postgres: logical replication
launcher
yuinishizawa      1743   0.0  0.0 411231232   3200   ??
Ss    5:41pm   0:00.29 postgres: autovacuum launcher
yuinishizawa      1742   0.0  0.0 411089920   1840   ??
Ss    5:41pm   0:00.13 postgres: walwriter
yuinishizawa      1740   0.0  0.0 411222016   1936   ??
Ss    5:41pm   0:00.14 postgres: background writer
yuinishizawa      1739   0.0  0.0 411099136   4736   ??
Ss    5:41pm   0:00.45 postgres: checkpointer
yuinishizawa      1733   0.0  0.0 411090240   7936   ??
S     5:41pm   0:01.50 /opt/homebrew/opt/postgresql@17/
bin/postgres -D /opt/homebrew/var/postgresql@17


# Base directory = /opt/homebrew/var/postgresql@17
$ ls /opt/homebrew/var/postgresql@17

PG_VERSION           pg_commit_ts         pg_ident.conf
pg_notify            pg_snapshots         pg_subtrans
pg_wal               postgresql.conf
base                 pg_dynshmem          pg_logical
pg_replslot          pg_stat              pg_tblspc
pg_xact              postmaster.opts
global               pg_hba.conf          pg_multixact
pg_serial            pg_stat_tmp          pg_twophase
postgresql.auto.conf postmaster.pid
```

==========
======

==Database==

```
CREATE DATABASE name
    [ WITH ] [ OWNER [=] user_name ]
           [ TEMPLATE [=] template ]
           [ ENCODING [=] encoding ]
           [ STRATEGY [=] strategy ]
           [ LOCALE [=] locale ]
           [ LC_COLLATE [=] lc_collate ]
           [ LC_CTYPE [=] lc_ctype ]
           [ BUILTIN_LOCALE [=] builtin_locale ]
           [ ICU_LOCALE [=] icu_locale ]
           [ ICU_RULES [=] icu_rules ]
           [ LOCALE_PROVIDER [=] locale_provider ]
           [ COLLATION_VERSION = collation_version ]
           [ TABLESPACE [=] tablespace_name ]
```

```
           [ ALLOW_CONNECTIONS [=] allowconn ]
           [ CONNECTION LIMIT [=] connlimit ]
           [ IS_TEMPLATE [=] istemplate ]
           [ OID [=] oid ]


ALTER DATABASE name [ [ WITH ] option [ ... ] ]

where option can be:

    ALLOW_CONNECTIONS allowconn
    CONNECTION LIMIT connlimit
    IS_TEMPLATE istemplate

ALTER DATABASE name RENAME TO new_name

ALTER DATABASE name OWNER TO { new_owner | CURRENT_ROLE
| CURRENT_USER | SESSION_USER }

ALTER DATABASE name SET TABLESPACE new_tablespace

ALTER DATABASE name REFRESH COLLATION VERSION

ALTER DATABASE name SET configuration_parameter { TO |
= } { value | DEFAULT }
ALTER DATABASE name SET configuration_parameter FROM
CURRENT
ALTER DATABASE name RESET configuration_parameter
ALTER DATABASE name RESET ALL


DROP DATABASE [ IF EXISTS ] name [ WITH FORCE]
```

==========
======

==Schema==

Doc: https://www.postgresql.org/docs/current/ddl-schemas.htm

Example: create and use new schema

```
# Check existing schema

> \dn
postgres=# \dn
       List of schemas
   Name   |       Owner
----------+-------------------
 public   | pg_database_owner
```

```
# 1) Create new schema
> CREATE SCHEMA myschema;

# 2) Set as default
> SET search_path TO myschema,public;

# 3) Use
> CREATE TABLE mytable (x integer);                #
Create table

> CREATE TABLE myschema.mytable (x integer);       #
Same explicit

> \d
postgres=# \d
                         List of relations
  Schema  |             Name             |   Type   |
Owner
----------+------------------------------+----------
+--------------
 myschema | mytable                      | table    |
yuinishizawa
```

==========
======

== Roles ==

Roles: https://www.postgresql.org/docs/current/user-manag.html
[Reference] SQL: https://www.postgresql.org/docs/current/sql-commands.html

SET ROLE: https://www.postgresql.org/docs/current/sql-set-role.html
Set role in current session
i.e. change **user**

```
SET [ SESSION | LOCAL ] ROLE role_name
SET [ SESSION | LOCAL ] ROLE NONE
RESET ROLE
```

CREATE ROLE: https://www.postgresql.org/docs/current/sql-createrole.html
DROP ROLE: https://www.postgresql.org/docs/current/sql-droprole.html
ALTER ROLE: https://www.postgresql.org/docs/current/sql-alterrole.html

```
CREATE ROLE name [ [ WITH ] option [ ... ] ]
where option can be:

    -- user
      SUPERUSER | NOSUPERUSER                  -- is
superuser            default=NO
    | CREATEDB | NOCREATEDB                    -- can
CREATE DATABASE  default=NO
```

```
    | CREATEROLE | NOCREATEROLE                -- can
CREATE ROLE      default=NO
    | LOGIN | NOLOGIN                          -- can
login                default=NO
    | REPLICATION | NOREPLICATION

    -- roles
    | INHERIT | NOINHERIT                      --
inherits from parent role         default=INHERIT
    | IN ROLE role_name [, ...]                -- GRANT
    | ROLE role_name [, ...]                   -- m
    | ADMIN role_name [, ...]

    | CONNECTION LIMIT connlimit               -- max #
of connections            default=0
                                               -- 0 = no
limit
    -- password
    | [ ENCRYPTED ] PASSWORD 'password'         --

    | PASSWORD NULL                            -- no
password     default

    -- misc
    | VALID UNTIL 'timestamp'
    | BYPASSRLS | NOBYPASSRLS
    | SYSID uid


ALTER ROLE role_specification [ WITH ] option [ ... ]
where option can be:
      SUPERUSER | NOSUPERUSER
    | CREATEDB | NOCREATEDB
    | CREATEROLE | NOCREATEROLE
    | INHERIT | NOINHERIT
    | LOGIN | NOLOGIN
    | REPLICATION | NOREPLICATION
    | BYPASSRLS | NOBYPASSRLS
    | CONNECTION LIMIT connlimit
    | [ ENCRYPTED ] PASSWORD 'password' | PASSWORD NULL
    | VALID UNTIL 'timestamp'

ALTER ROLE name RENAME TO new_name

DROP ROLE [ IF EXISTS ] name [, ...]
```

GRANT ROLE: https://www.postgresql.org/docs/current/sql-grant.html
REVOKE ROLE: https://www.postgresql.org/docs/current/sql-revoke.html

```
-- add to roles aka add user to group
GRANT role_name [, ...] TO role_specification [, ...]
    [ WITH { ADMIN | INHERIT | SET } { OPTION | TRUE |
FALSE } ]
```

```
    [ GRANTED BY role_specification ]


-- remove from roles
REVOKE [ { ADMIN | INHERIT | SET } OPTION FOR ]
    role_name [, ...] FROM role_specification [, ...]
    [ GRANTED BY role_specification ]
    [ CASCADE | RESTRICT ]

where role_specification can be:

    [ GROUP ] role_name
  | PUBLIC
  | CURRENT_ROLE
  | CURRENT_USER
  | SESSION_USER
```

ROLE options
[Reference] CREATE ROLE: https://www.postgresql.org/docs/current/sql-createrole.html
PostgreSQL has additional options

```
# User options
[NO]SUPERUSER           Superuser. Default=NO
[NO]CREATEDB            Allowed to create new database. Default=NO
[NO]CREATEROLE          Allowed to create new roles. Default=NO
[NO]LOGIN               Can have login session. Default=NO
[NO]REPLICATION

# Inheritance
[NO]INHERIT             Inherits from parent role. Default=INHERIT

# password
PASSWORD NULL;          No password. Default.
PASSWORD 'aaa'
ENCRYPTED PASSWORD 'aaa'

# connection
CONNECTION LIMIT -1     No connection limit. Default
CONNECTION LIMIT 10     Max
```

Role options vs privileges vs configurations

role options CREATE ROLE yui WITH ...
Admin permissions
ALTER ROLE yui WITH **..**
e.g. CREATEROLE
DROP ROLE yui
LOGIN

role privilege GRANT ALL ON DATABASE mydb TO yui
Object-level permissions
REVOKE ALL ON DATABASE mydb FROM yui
e.g. can SELECT, UPDATE a given table

role configuration ALTER yui IN DATABASE mydb SET ...
role **settings** and variables
aka. **parameter**
e.g. search_path

Example: create user and assign to schema

Step 1) Create admin user and schema

```
DROP SCHEMA IF EXISTS abcschema CASCADE;
DROP ROLE IF EXISTS abcadmin, abcuser;

-- user: admin
CREATE ROLE abcadmin
  WITH
    CREATEROLE
    LOGIN
    PASSWORD 'abc';
ALTER ROLE abcadmin SET search_path=abcschema;

-- create schema
CREATE SCHEMA abcschema
  AUTHORIZATION abcadmin

  CREATE TABLE abctable (
    id INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    x INT NOT NULL,
    y INT NOT NULL DEFAULT 10,
    z INT GENERATED ALWAYS AS (y * x) STORED
  );
```

Step 2) Create normal user

```
-- user: normal user
CREATE ROLE abcuser
  WITH
    LOGIN
    PASSWORD 'abc';

-- set default schema
ALTER ROLE abcuser SET search_path=abcschema;
-- allow read access to schema by default
GRANT USAGE ON SCHEMA abcschema TO abcuser;
-- set permissions
GRANT SELECT, INSERT(x), UPDATE(x) ON abcschema.abctable
TO abcuser;
```

Step 3) Access as normal user

```
> INSERT INTO abctable (x) VALUES (1), (3);
INSERT 0 2
```

```
> SELECT * FROM abctable;
 id | x | y  | z
════╪═══╪════╪════
 1 | 1 | 10 | 10
 2 | 3 | 10 | 30
(2 rows)

> INSERT INTO abctable (x,y) VALUES (1,2);
permission denied

> DELETE FROM abctable WHERE id=1;
permission denied
```

Privileges

PostgreSQL privileges

GRANT: https://www.postgresql.org/docs/current/sql-grant.html
[PostgreSQL] 5.8 Privileges: https://www.postgresql.org/docs/current/ddl-priv.html

GRANT ROLE: https://www.postgresql.org/docs/current/sql-grant.html
REVOKE ROLE: https://www.postgresql.org/docs/current/sql-revoke.html

```
-- Table privileges
GRANT { { SELECT | INSERT | UPDATE | DELETE | TRUNCATE |
REFERENCES | TRIGGER | MAINTAIN }
    [, ...] | ALL [ PRIVILEGES ] }
    ON { [ TABLE ] table_name [, ...]
         | ALL TABLES IN SCHEMA schema_name [, ...] }
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Table column privileges
GRANT { { SELECT | INSERT | UPDATE | REFERENCES }
( column_name [, ...] )
    [, ...] | ALL [ PRIVILEGES ] ( column_name
[, ...] ) }
    ON [ TABLE ] table_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Sequence privileges
GRANT { { USAGE | SELECT | UPDATE }
                                    -- SELECT = currval
    [, ...] | ALL [ PRIVILEGES ] }
                                    -- USAGE  = currval,
nextval
    ON { SEQUENCE sequence_name [, ...]
                                    -- UPDATE = nextval,
setval
         | ALL SEQUENCES IN SCHEMA schema_name [, ...] }
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]
```

```
-- Database privileges
GRANT { { CREATE | CONNECT | TEMPORARY | TEMP } [, ...]
| ALL [ PRIVILEGES ] }            -- CREATE =  can
create e.g. CREATE SCHEMA
    ON DATABASE database_name [, ...]
                                  -- CONNECT = read only
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Function privileges
GRANT { EXECUTE | ALL [ PRIVILEGES ] }
    ON { { FUNCTION | PROCEDURE | ROUTINE } routine_name
[ ( [ [ argmode ] [ arg_name ] arg_type [, ...] ] ) ]
[, ...]
         | ALL { FUNCTIONS | PROCEDURES | ROUTINES } IN
SCHEMA schema_name [, ...] }
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Domain privileges
GRANT { USAGE | ALL [ PRIVILEGES ] }
    ON DOMAIN domain_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Foreign privileges
GRANT { USAGE | ALL [ PRIVILEGES ] }
    ON FOREIGN DATA WRAPPER fdw_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]
GRANT { USAGE | ALL [ PRIVILEGES ] }
    ON FOREIGN SERVER server_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Language privileges
GRANT { USAGE | ALL [ PRIVILEGES ] }
    ON LANGUAGE lang_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Large object privileges
GRANT { { SELECT | UPDATE } [, ...] | ALL
[ PRIVILEGES ] }
    ON LARGE OBJECT loid [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

GRANT { { SET | ALTER SYSTEM } [, ... ] | ALL
[ PRIVILEGES ] }
    ON PARAMETER configuration_parameter [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]
```

```
-- Schema privileges
GRANT { { CREATE | USAGE } [, ...] | ALL
[ PRIVILEGES ] }                   -- USAGE = read only
e.g. can't CREATE TABLE
    ON SCHEMA schema_name [, ...]
                       -- CREATE= write
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

-- Tablespace privileges
GRANT { CREATE | ALL [ PRIVILEGES ] }
    ON TABLESPACE tablespace_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

GRANT { USAGE | ALL [ PRIVILEGES ] }
    ON TYPE type_name [, ...]
    TO role_specification [, ...] [ WITH GRANT OPTION ]
    [ GRANTED BY role_specification ]

where role_specification can be:

    [ GROUP ] role_name
  | PUBLIC
  | CURRENT_ROLE
  | CURRENT_USER
  | SESSION_USER
```

REVOKE ROLE: https://www.postgresql.org/docs/current/sql-revoke.html

```
REVOKE [ GRANT OPTION FOR ]
    { { SELECT | INSERT | UPDATE | DELETE | TRUNCATE |
REFERENCES | TRIGGER | MAINTAIN }
    [, ...] | ALL [ PRIVILEGES ] }
    ON { [ TABLE ] table_name [, ...]
         | ALL TABLES IN SCHEMA schema_name [, ...] }
    FROM role_specification [, ...]
    [ GRANTED BY role_specification ]
    [ CASCADE | RESTRICT ]

REVOKE [ GRANT OPTION FOR ]
    { { SELECT | INSERT | UPDATE | REFERENCES }
( column_name [, ...] )
    [, ...] | ALL [ PRIVILEGES ] ( column_name
[, ...] ) }
    ON [ TABLE ] table_name [, ...]
    FROM role_specification [, ...]
    [ GRANTED BY role_specification ]
    [ CASCADE | RESTRICT ]

... similar to GRANT ROLE
```

Privileges
In addition to SQL standard privileges, PostgreSQL has additional privileges

Targets:

[standard SQL]
TABLE
SEQUENCE
...

[Postgres]
DATABASE
SCHEMA
FUNCTION
ROUTINE
...

Privileges:

[standard SQL]
SELECT
SELECT (col1, col2)
INSERT
INSERT (col1, col2)
...

[Postgres]

USAGE	schema: allow access to objects in schema
TRUNCATE	
CREATE	database: allow creation of schema schema: allow creation of objects e.g. tables
CONNECT	Allow connection
SET	Allow using SET

...

==========
======

== COPY and \copy==

COPY

```
COPY table_name [ ( column_name [, ...] ) ]
    FROM { 'filename' | PROGRAM 'command' | STDIN }
    [ [ WITH ] ( option [, ...] ) ]
    [ WHERE condition ]

COPY { table_name [ ( column_name [, ...] ) ] |
( query ) }
    TO { 'filename' | PROGRAM 'command' | STDOUT }
    [ [ WITH ] ( option [, ...] ) ]

where option can be one of:

    FORMAT format_name
    csv|text|binary  default:text
```

```
    FREEZE [ boolean ]
    VACUUM FREEZE
    DELIMITER 'delimiter_character'
    column delimiter

    text='\t'    tab

    csv=','      comma
    NULL 'null_string'
    null representation

    text='\N'

    csv=''       empty string
    DEFAULT 'default_string'
    HEADER [ boolean ]
    file includes header
    QUOTE 'quote_character'
    '"'          double quote
    ESCAPE 'escape_character'

    -- COPY FROM only
    HEADER MATCH
    # of columns in header must match # of columns in
data
    FORCE_NOT_NULL { ( column_name [, ...] ) | * }

    FORCE_NULL { ( column_name [, ...] ) | * }


    -- COPY TO only
    FORCE_QUOTE { ( column_name [, ...] ) | * }
    always quote



    ON_ERROR error_action
    ignore|stop
    ENCODING 'encoding_name'
    LOG_VERBOSITY verbosity
    default|verbose
```

COPY vs \copy command
COPY uses the **server's file system**.
\copy uses the **client's file system**.

*** usage: copy FROM**
See man psql

e.g.
Step) prepare **data file** to import
file.csv

```
id,value
1,abc
2,def
```

Step) prepare **TABLE**

```
CREATE TABLE t1 (
  id INT NOT NULL PRIMARY KEY,
  comment TEXT NOT NULL
);
```

case 1: with **filename option**
\copy uses the **client's file system**
COPY uses the **server's file system**

```
> \copy t1 FROM './file.csv' WITH (FORMAT 'csv', HEADER
FALSE);      -- OK    Uses file.csv on client
> COPY t1 FROM './file.csv' WITH (FORMAT 'csv', HEADER
FALSE);      -- BAD?  Uses file.csv on server
```

case 2: with **STDIN option** + **file**

```
$ psql ...connection options... -c "\copy t1 FROM STDIN
WITH (FORMAT 'csv', HEADER TRUE)"  < file.csv
     -- OK  --command + \copy
COPY 2

# Same with tail
$ tail -n +2 file.csv | psql  ...connection options...
-c "\copy t1 FROM STDIN WITH (FORMAT 'csv', HEADER
FALSE)"


# GOTCHA: doesn't seem to work with --file option
# WHY?
$ cat copyscript.sql

\copy t1 FROM STDIN WITH (FORMAT 'csv', HEADER TRUE);
$ psql ...connection options... -f copyscript.sql  <
file.csv                  -- BAD Didn't copy
COPY 0
```

case 3: **STDIN option** + key input

a) in **psql session**

```
> \copy t1 FROM STDIN WITH (FORMAT 'csv', HEADER
FALSE);                        -- OK  With \copy
```

```
Enter data to be copied followed by a newline.
End with a backslash and a period on a line by itself,
or an EOF signal.
>> 1,a
>> 2,b
>> \.
COPY 1

> COPY t1 FROM STDIN WITH (FORMAT 'csv', HEADER
FALSE);                        -- OK  With COPY
...
```

*** usage: copy TO**

case 1: **STDOUT option**
Can use either **copy** or **COPY**

```
> \copy t1 TO STDOUT WITH (FORMAT 'csv', HEADER TRUE);
             -- OK with \copy
id,comment
1,a
2,b

> COPY t1 TO STDOUT WITH (FORMAT 'csv', HEADER TRUE);
             -- OK with COPY
```

Pipe to file

```
# -X|--no-psqlrc      suppress psqlrc
psql ... -X -c "COPY t1 TO STDOUT WITH (FORMAT 'csv',
HEADER TRUE)"     > out.csv
```

case 2: **filename option**

\copy	saves on **client file system**
	may be **relative path**
COPY	saves on **server file system**
	MUUST be **absolute path**

```
> \copy t1 TO './out.csv' WITH (FORMAT 'csv', HEADER
TRUE)"                 -- OK with \copy      out.csv on
client file system

> COPY t1 TO './out.csv' WITH (FORMAT 'csv', HEADER
TRUE)"                 -- ERROR MUST be absolute path
> COPY t1 TO '/tmp/out.csv' WITH (FORMAT 'csv', HEADER
TRUE)"            -- OK with COPY       /tmp/out.csv on
server file system
```

*** GOTCHA: COPY TO with in text format escapes newline as '\n'**
Problem: text with newline such as '\n' get escaped on output.
This is because **each row** MUST be a **single line**.
Thus, new lines within a **column** are escaped.

```
-- newline normally
> VALUES (E'a\nb');
a
b

-- BAD: new line in column
> \copy (VALUES (E'a\nb')) TO /tmp/out.txt WITH (FORMAT
text);
> \! cat /tmp.out.txt
a\nb

-- GOOD: separate rows
>  \copy (VALUES ('a'), ('b')) TO /tmp/out.txt WITH
(FORMAT text);
> \! cat /tmp.out.txt
a
b
```

*** GOTCHA: \copy fails if multiline**
GOCHA: \copy does not support variables
Ref: https://www.postgresql.org/docs/current/app-psql.html#APP-PSQL-META-COMMANDS-COPY

Problem: \copy is a **PQSL command**, not **SQL**.
Hence it has some limitations, such as single line and no variables.

Solution: try to use COPY instead

```
\set val 'a'

-- case: \copy
\copy (VALUES ('a')) TO 'tmp/out.txt' WITH (FORMAT
text);       -- OK since line
\copy (VALUES (:'val')) TO 'tmp/out.txt' WITH (FORMAT
text);   -- ERROR No variables
\copy (
         -- ERROR No multiline
  VALUES ('a')
) TO 'tmp/out.txt' WITH (FORMAT text);
\copy (VALUES ('a')) TO 'tmp/out.txt' WITH (FORMAT
text);


-- case: COPY
COPY (
         -- OK
  VALUES (:'val')
) TO STDOUT WITH (FORMAT text) \g 'tmp/out.txt';
```

*** GOTCHA: COPY FROM file fails with error "exit code 1"**

e.g.

```
> COPY mytable FROM 'cat /tmp/data.csv' with (FORMAT
'csv', HEADER TRUE);
DETAIL:  child process exited with exit code 1
```

Problem: the command could not read the file.
Possible causes are
1) f**ile location**: file must be located on the **server**, not the client.
2) **file permission**: file must be have at least **read** permission for the postgres user

```
# Make sure the file exists on the server
$ ls /tmp/
-r--------    1 501      dialout     494199 Apr 23
04:06 /tmp/data.csv

# Make sure postgres has read permission at least
$ chmod go+r /tmp/data.csv
```

Solution:
COPY from file has several requirements
1) **file location:** file must be on the **server**, not the client.
2) **file permission**: file must be readable by postgres
3) PostgreSQL **ROLE**: the user must either
  a) have **pg_read_server_files** to read the file and **pg_execute_server_program**
  b) be **superuser**

```
psql:load.sql:92: ERROR:  must be superuser or a member
of the pg_execute_server_program role to COPY to or from
an external program
HINT:  Anyone can COPY to stdout or from stdin. psql's
\copy command also works for anyone.
```

==========

== OLD ==